InterVox

Project report — how we built an AI practice interviewer

2026-10-06 → 2026-10-13 · live at commit 0c71871 · https://intervox-pi.vercel.app

Covers 2026-10-06 (first commit) to 2026-10-13 (live at commit `0c71871`).

Sources. This report uses only the project's own records: the git history (122 commits), `CLAUDE.md`, the `docs/` folder (`SPEC.md`, `INTERVIEWER_RULES.md`, `RULES_CHECK.md`, `DEPLOY.md`), the ticket table `orchestration.tickets` (114 InterVox tickets) and the code. A claim that comes from a ticket or commit cites it as **T-n** or `commit`. Anything the sources don't settle is marked *Unclear*.

---

1. Overview

What it is. InterVox is a practice interviewer for any job. The user uploads a résumé and a job description and picks a length: 10, 30, 45 or 60 minutes, or a custom length from 5 to 60. They then answer the interviewer's questions out loud or by typing, and get a scored report with a "stronger version" of each answer rewritten from their own facts. (`docs/SPEC.md`, `CLAUDE.md`)

Who it's for. "Anyone job-hunting, in any field — software, sales, marketing, media, business, healthcare, retail, and more" (`docs/SPEC.md`). New fields are handled by mixing reusable interview *format modules*, such as behavioral, technical, coding, system design, role-play and case questions, rather than by writing new code for each field (`CLAUDE.md`).

The cost rule. `CLAUDE.md` hard rule 1: *"Free tiers and open source everywhere, except Fire API usage (transcribe, chat, speech). Never add another paid service or dependency."* In practice:

- **Fire API** is the only paid service. It handles speech-to-text, the interviewer's replies and scoring, and the optional voice. The backend enforces a **total Fire budget of $1 per month** (`FIRE_MONTHLY_BUDGET_USD=1`). New sessions pause when it runs out.
- For October 2026 only, the live site's budget is **$1.45** (T-76, `CLAUDE.md`). It goes back to $1 in November.
- The spec also lists the developer's Claude Pro subscription as already paid (`docs/SPEC.md`).
- Every other piece is free: open-source libraries, Vercel Hobby, DuckDNS and Let's Encrypt.

Framing. Scores are practice feedback, not an assessment. A disclaimer is shown for licensed or specialized professions (hard rule 6).

The six hard rules (`CLAUDE.md`):

1. Minimal cost.
2. The Fire key never leaves the backend.
3. Never store audio or video.
4. Never invent experience in rewrites.

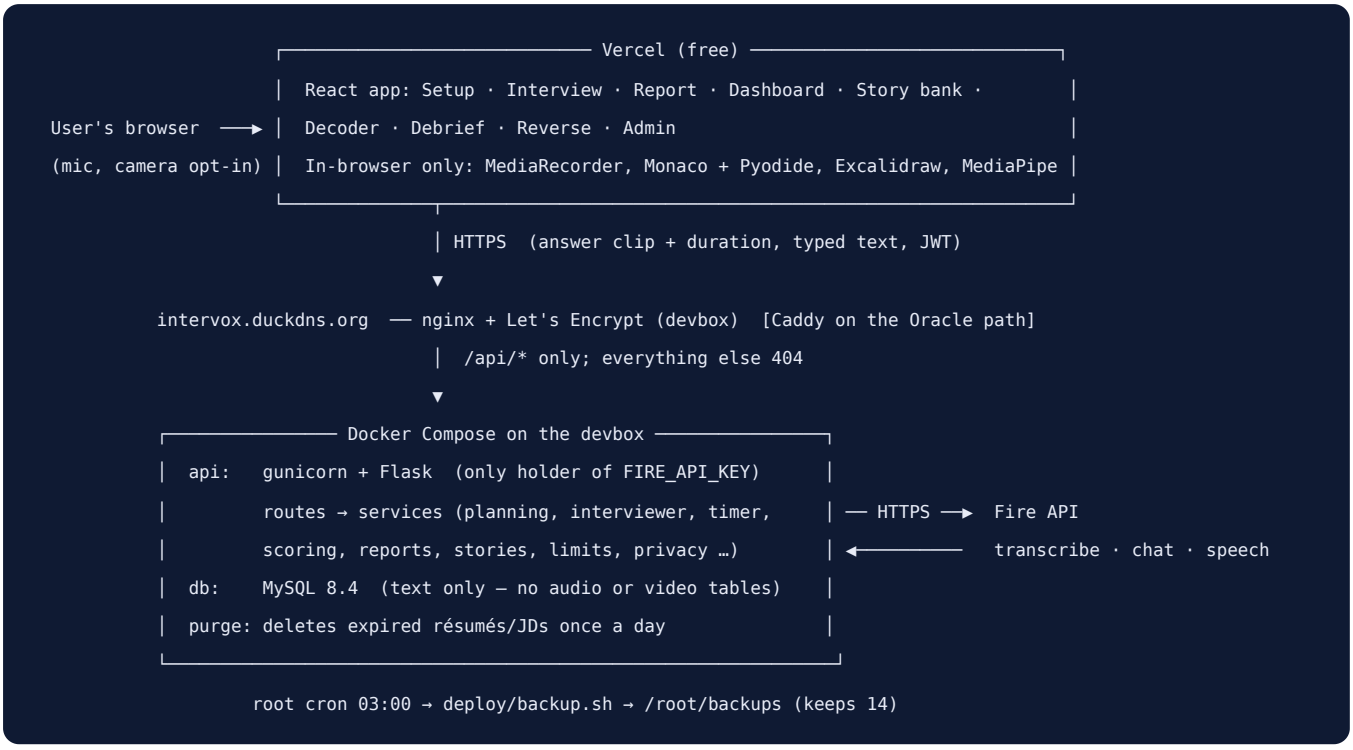
5. Stay respectful at every pressure level.
6. Scores are practice feedback.

2. Tech stack

Layer	Tool / library / service	Why it was chosen (source)
Frontend	**React 19**, **Vite 6**, **Tailwind CSS 4**, React Router 7	Free and open source; `CLAUDE.md` stack table.
Recording	Browser **MediaRecorder API** (webm/opus)	Records each answer in the browser at no cost. Clips go to Flask and are discarded after transcription (hard rule 3).
Code editor	**Monaco** (`@monaco-editor/react` , `monaco-editor`), bundled as ES modules	Coding rounds. Loading Monaco from the CDN broke Excalidraw and MediaPipe, so it is bundled locally (**T-51**).
Run code	**Pyodide** (Python) and a JavaScript runner, both in a Web Worker	The "Run" button runs code in the user's browser, with a 5 s limit and nothing run on the server (**T-84**).
Whiteboard	**Excalidraw**	System design rounds. The sketch is sent to the AI as text (`CLAUDE.md`).
Body language	**MediaPipe Tasks Vision**	Optional camera check that runs only in the browser. Only percentages are saved, never video (`RULES_CHECK.md` , Phase 9).
Interviewer voice	Browser **Web Speech API** by default; Fire `POST /v1/speech` (`tts-kokoro`) when `VOICE_PROVIDER=fire`	Browser voice is free. Fire speech costs about $0.0003 per short reply and stays off by default (**T-27**).
Backend	**Flask 3.1**, Flask-SQLAlchemy, Flask-Migrate (Alembic), **Flask-JWT-Extended**, flask-cors, **Pydantic 2**, requests, pypdf, python-dotenv, **gunicorn**	Flask is the only component that talks to Fire (hard rule 2). JWT handles authentication. Pydantic validates every LLM reply. pypdf reads uploaded résumés.
Speech-to-text	Fire `POST /v1/transcribe` (Whisper Large v3 Turbo)	Under 1 cent per minute (`SPEC.md`). It keeps "um" and "uh", so filler counts work (verified live 2026-10-09, `CLAUDE.md`).
AI brain	Fire `POST /v1/chat` , model `gpt-4o-mini` (`FIRE_CHAT_MODEL`)	Handles every interviewer turn, plan, rubric, score and rewrite. Chosen because Fire's default model, Claude Sonnet, "is far pricier" (`CLAUDE.md`).
Database	**MySQL 8** (dev), **MySQL 8.4** container (live)	Free (`SPEC.md`). Tests use in-memory SQLite with foreign keys enforced like MySQL (`commit 7191a44`).
Hosting (planned)	Oracle Cloud Always Free VM (2 cores / 12 GB ARM), Docker Compose, Caddy	Free tier (`SPEC.md`). Oracle refused the sign-up, so this was never used (see §7).
Hosting (actual)	The **devbox** server: Docker Compose (db, api, purge) behind **nginx + certbot**; frontend on **Vercel Hobby**	Free. nginx already owned ports 80/443 on the devbox (`docs/DEPLOY.md` §2b, `commit 27501f6`).
Domain / HTTPS	**DuckDNS** (`intervox.duckdns.org`), Let's Encrypt via certbot (Caddy on the Oracle path)	Free subdomain with automatic HTTPS. Microphone access requires HTTPS (`DEPLOY.md`).

Layer	Tool / library / service	Why it was chosen (source)
Testing	pytest (406 backend tests), Vitest 5 + Testing Library (107 frontend tests), black, ESLint	`CLAUDE.md` convention: every feature gets a test, and Fire is always mocked. Vitest went from version 3 to 5 to clear npm audit findings (**T-54**).
Evals	`evals/run.py`: AI candidates (strong, weak and rambling) run through the real API with a simulated clock and a budget guard	Measures whether scores separate good answers from weak ones (`commit c8cc4d4` , **T-72–T-75**).
Coordination	`pm` CLI and `orchestration.tickets` on the devbox; a separate "master" agent session deploys	Every problem gets a ticket. Deploys go through a human approval gate (`/home/hibahnafees/CLAUDE.md`).
Optional, not built	LiveKit Agents (real-time voice), Hume EVI (emotion experiment)	Phases 11–12 in `CLAUDE.md` ; listed in the spec as optional.

3. Architecture



One interview turn (`CLAUDE.md` , "Core concepts"):

1. The browser uploads the answer clip and the duration it measured.
2. Flask calls Fire *transcribe* and saves only the text.
3. Flask calls Fire *chat* with the timer state.
4. Flask returns the interviewer's next message: text, plus audio when Fire voice is on.

The **timer lives in the backend**, not in the AI. Each chat call gets the elapsed and remaining time for every section, so the interviewer moves on or wraps up between answers.

Reports are generated after the session in a background thread, one Fire chat call per question:

- That call scores the answer and writes the stronger version.

- Code then runs extra checks on each question: a gap check (**T-72**), a second opinion before an answer is scored 1 (**T-112**), and grounding of the rewrite (**T-15/T-18/T-111**).
- One more call writes the verdict and the top three takeaways.
- Weights, thresholds and delivery analytics (pace, filler words, time used against the budget) are computed in Python.

Code layout (from the repo):

- `backend/app/services/` has one service per job: `fire.py`, `planning.py`, `interviewer.py`, `timer.py`, `scoring.py`, `reports.py`, `stories.py`, `limits.py`, `privacy.py`, `pressure.py`, `panel.py`, `negotiation.py`, `debriefs.py`, `reverse.py` and others.
- `backend/app/prompts/` holds the 25 prompt templates, which are never written inline.
- `frontend/src/pages/` and `frontend/src/components/` hold the React app.

All Fire calls go through one client. It has a 60 s timeout and retries once, on a 5xx error or a network failure only (`services/fire.py`).

---

4. Timeline

The work happened in 6 days with commits: Oct 6 (2 commits), Oct 8 (1), Oct 9 (29), Oct 10 (30), Oct 11 (29) and Oct 12 (31).

Date	Phase / step	What was built	Key commits	Tickets
10-06	Spec	Project spec, `CLAUDE.md`, `docs/SPEC.md`; panel and negotiation modes added	`f4e5b27`, `e2e353f`	—
10-08	Spec	Switched to the Fire API design with a $1/month budget	`611da26`	—
10-09	**1. Foundation**	Flask API, MySQL schema, JWT auth, React shell	`0e5eef3`, `5afb04f`	T-12
10-09	**2. Text-only brain**	Documents, JD analysis, plan, rubric, typed turns; setup and plan editor	`d71a3d6`, `a7cad0a`, `e561cf2`, `2393751`	T-13, T-14
10-09	**3. Speak in, text out** (first release)	Recorder, silence auto-stop, Fire transcribe, backend timer	`4bf77ad`	—
10-09	**4. Report**	Scores, stronger versions, delivery analytics, Try again; rewrite grounding	`38d3c7d`, `47a4098`, `7c94e52`, `2985974`	T-15–T-19
10-09	**5. Any job**	Format modules, interview types, role-play, all checked live	`392e51b`, `f8fd775`… `be02697`	T-20–T-26
10-09	**6. Turn-based voice**	Speaker segments, browser voice, Fire speech switch; polish: ice-breaker, sign-offs, stats dashboard, redesign, setup choices, "any questions for me?"	`8ff30a8`, `172308b`, `dce5656`, `7fee9a6`, `0ff7c94`, `235edf6`	T-27, T-29–T-39
10-10	Rules check	`INTERVIEWER_RULES.md` and the after-every-phase check; fixes for every gap; first live re-check	`6c15f1d`, `192e407`, `3501012`, `9f1dda1`	T-40–T-48
10-10	**7. Progress and story bank**	Story extraction and merging, question bank with spaced repetition, weak-spot interviews, weekly goal	`5112874`, `3971e7f`, `8a7cd1b`, `dbcf6c3`	—

Date	Phase / step	What was built	Key commits	Tickets
10-10	**8. Signature modes**	JD decoder (works signed out), pressure mode (levels 1–5), debrief (recall only), reverse mode	`f19ea36`, `864337c`, `ea76739`, `71d2ca7`, `8c4436d`	T-49
10-10	**9. Extras**	Gap analysis, questions-for-them coach, cheat sheet, thank-you email, company research, panel, negotiation, coding (Monaco), whiteboard (Excalidraw), body language (MediaPipe)	`fc134f3`…`daec68b`, `7070d6f`, `0aab033`, `17f1eb6`	T-51–T-65
10-10	Scorer fixes	Quoted results credited; job criteria scored when the question tests them	`ee16636`, `833c10c`	T-66, T-68–T-71
10-11	**10. Deploy and harden**	npm audit cleared; Fire speech pinned; privacy controls; admin page; rate limits and production checks; Docker Compose and Caddy; eval harness; deploy guide and backup script	`d2b5369`, `4b26a13`, `7191a44`, `91b6f2f`, `d3427b2`, `5ea6dd4`, `c8cc4d4`, `639a3fa`, `b48ca1f`	T-27, T-54, T-72–T-75
10-11	**Go-live**	Oracle sign-up failed, so the API went to the devbox behind nginx; renamed to **InterVox**; dark mode; backup fix	`27501f6`, `e5498a8`, `e077547`, `51b4610`, `88d21ab`	T-76–T-80
10-11	First live feedback	Code was lost with spoken answers; hands-on rounds were too short; Run button; 1–10 problems; favicon; recorder fixes	`05cbc38`, `3b909ed`, `7350137`, `a1ebdc0`	T-81–T-88
10-12	Quality passes	Planning (coding and design offered for software jobs, no repeated problems), five more eval fields, gap check, pressure-mode questions, interviewer personas	`88cdc34`…`964b39a`	T-89–T-97
10-12	Quality passes	Small talk, off-topic steering, coding difficulty per level, debrief fixes, reports that always finish	`1bebc2d`…`2583cca`	T-98–T-106
10-12	Live checks	Results kept in rewrites, time-up cut-in, gap fit, invented links, off-subject stories, pressure tips	`5df3806`…`40c76f4`	T-107–T-116
10-12	Live checks	Praise filter, summaries, "Thanks" after a cut-off, metric words, leaked token, deleted-account tokens, rewrite repairs	`02e9a54`…`0c71871`	T-117–T-130

Every server deploy after go-live went through a ticket with a deploy plan, a human `pm approve`, and the master agent pulling and rebuilding (§7).

5. Features built (what works today)

Interview setup

- Upload or paste a résumé and job description; PDF text is read with pypdf.
- JD analysis finds the field, seniority, competencies and which format modules fit.
- The question plan has per-section minute budgets that add up to the chosen length, plus a reserve of bonus questions. The user can edit it.
- Durations: 10/30/45/60 minutes or custom. The user can review the questions first or start right away, and answer by speaking or typing (**T-38**).

- Interview types and hands-on rounds: 1–10 coding problems and 1–10 design questions, with any interview type (**T-85, T-95**).
- Persona: friendly, neutral or tough, each with its own manner (**T-97**). Difficulty: entry, mid or senior.

During the interview

- Ice-breaker small talk that is never scored (**T-34**), then one question at a time with follow-ups.
- The interviewer never praises answers (**T-14**) and steers off-topic answers back to the question (**T-100**).
- Controls: hint, skip, pause/resume and repeat (**T-40**).
- Recorder with silence auto-stop, an undo window and Cancel (**T-32, T-88**). Answers are capped at 5 minutes.
- Interviewer voice: browser speech, or Fire speech (`tts-kokoro`) when switched on.
- At the end, "Do you have any questions for me?", answered in role (**T-39**).

Formats and modes

- Behavioral, résumé deep-dive, technical, case, scenario, role-play, portfolio, coding (Monaco plus Run) and system design (Excalidraw).
- **Pressure mode**, levels 1–5:
  - levels 3–5 give 60/45/30 seconds per answer, and the candidate can ease off;
  - the report shows a composure score;
  - when time runs out the interviewer cuts in politely and the partial answer is saved (**T-109**).
- **Panel** (2–3 interviewers) and **salary negotiation** (a recruiter role-play).
- **Reverse mode**: the user interviews an AI candidate with a hidden profile.
- **Debrief**: recall a real interview from memory, have it scored, and get the next round predicted. Recordings are never uploaded (hard rule 3).
- **Body language**: opt-in, measured in the browser, only percentages saved.

Report

- Weighted score: role criteria 70%, communication 30% (`planning.share_weights`, **T-75**).
- Verdict, top takeaways and recommended exercises.
- Per question: criteria with comments, STAR check, gaps, weak moments, "what you did well", and the AI's confidence.
- Stronger version built only from the user's own facts, with placeholders where facts are missing (hard rule 4).
- Delivery analytics: speaking pace, filler words, time used against the budget.
- **Try again**: a retry scored on the same criteria as the first attempt (**T-29**), capped by `DAILY_RETRIES`.
- Read-only share links that can be revoked; cheat sheet and thank-you email draft.

Progress

- Dashboard: score over time, level, streak, roles and skills (**T-36**).
- Story bank built from answers, checked against the source, with duplicates merged.
- Question bank with spaced repetition, weak-spot interviews and a weekly goal.

Other tools

- JD decoder that works signed out, with saved decodes (**T-64**).

- Résumé gap analysis, a coach for the questions to ask them, and company research that fetches the public site safely.

Accounts, privacy and admin

- JWT auth with rate limits on login, signup and delete.
- **Delete my data**: Fire spend is kept in an anonymous tally, and old tokens are refused straight away (**T-125**).
- Résumés and JDs are deleted after 30 days unless saved (**T-103**).
- Admin usage page (`ADMIN_EMAILS`).
- Light theme by default with a dark switch (**T-78, T-86**).

---

6. Challenges and fixes

The pattern behind most fixes: **gpt-4o-mini treats prompt rules as soft** (**T-14** root cause). Rules that matter end up enforced in code, with the prompt kept as a hint.

6.1 The debrief scoring bug (T-105, T-106)

- **Symptom:** a debrief report stuck at "10 of 11 scored". Clicking *Finish the report* failed the same way every time.
- **Cause:**
  - The candidate's recalled *"Do you have any questions for us?"* was scored like an answer. The scorer then returned an empty stronger version, which the schema (minimum length 1) rejected.
  - One question that could never be scored blocked the whole report.
- **Fix:**
  - `f56c808`: reports finish even when an answer can't be scored. Bad entries are trimmed or dropped at every level, and a question that is unusable in two runs is set aside and shown as "not scored".
  - `bce734a`: a recalled "any questions for us?" is rated as the candidate's own questions, and an empty rewrite falls back to the résumé.
  - **T-106** (`2583cca`): the candidate's questions are matched by words, so a question split from a longer sentence is still rated.
  - Related: **T-93**, where one over-long answer had failed the whole report.

6.2 Pressure-mode timing (T-96, T-99, T-109, T-116, T-119, T-130)

- **T-96/T-99:** pressure mode didn't plan pressure questions or push back, and it moved on from vague answers silently. Fixed in `964b39a` and `1bebc2d`.
- **T-109:** when an answer's time ran out, the partial answer was lost and the cut-off was abrupt. Fixed in `fe02c0a`:
  - the browser says a polite line from a fixed list the server accepts;
  - the partial answer is saved with `answers.cut_off` (new migration) and scored with a "ran out of time" tip;
  - the reply moves on without a follow-up.
- **T-116** (`40c76f4`): cut-off and time-limited answers also got *"This was short (30 s of a 5.0 min budget)"*. That tip is now skipped when an answer was under a time limit.

- **T-119** (`bee1f17`): a fixed move-on line said *"Thanks. Let's move on."* right after the cut-in. It is used when the model asks a follow-up that isn't allowed, and it skipped the no-thanks rule.
- **T-130** (`0c71871`): an unfinished but relevant answer scored Relevance 2, so the rewrite became a generic template. A cut-off answer is no longer treated as off-topic.

6.3 The backup script (T-79, T-80)

- **Symptom:** the first root cron run of `deploy/backup.sh` worked (T-79), but master noted two problems:
  - with `sh` and no `pipefail`, a failed `mysqldump` piped into `gzip` still left a valid, tiny `.gz` and exited 0;
  - the `mock` user caused a PROCESS-privilege error on every run.
- **Fix** (`88d21ab` , **T-80**):
  - bash with `set -euo pipefail` ;
  - the dump goes to a `.partial` file that is deleted on failure, so a failed dump never replaces a good backup;
  - `--no-tablespaces` , plus a log line on every run.
- Master's re-run: exit 0, 9127 bytes, `gzip -t` OK, 19 tables.

6.4 Deployment blocks

- **Oracle Cloud refused the sign-up**, so the planned free VM was never available (`27501f6` , `DEPLOY.md` §2b).
  - The API runs on the devbox instead, where nginx and certbot already held ports 80/443.
  - Caddy is turned off by `deploy/compose.devbox.yml` , the API listens only on `127.0.0.1:8000` , and an nginx site proxies `/api/` .
- **Deploys are human-gated.** The building agent was not allowed to run production deploys or read the production database directly (`CLAUDE.md` : *"Server deploys go through master"*).
  - Each fix becomes a ticket in `pending_review` with a JSON deploy plan (steps, commit, covers, rollback).
  - The user runs `pm approve T-n` , and the master agent pulls, rebuilds the `api` container and closes the ticket.
- **The live server falling behind main.** Main had moved on after the first deploy (T-76), and **T-81** (*"hands-on rounds too short"*) turned out to be the live server still running code from before the planning fixes.

6.5 Interview behaviour

- **Praise and ignoring off-topic replies (T-14).**
  - Code filters evaluative sentences (`neutralize`).
  - **T-117** later changed the filter so that praise of a résumé item becomes *"Let's talk about …"* , because dropping the whole sentence left "that project" with nothing to refer to.
- **Repeated follow-ups (T-44, T-70):** a check in code catches near-repeats and moves on, and the prompt lists the earlier follow-ups.
- **Role-play issues (T-20–T-26):** the plan failed when the scenario object ended up in the question list, the character never accepted a next step, and the next question leaked into the character's lines.
  - The character now returns `goal_reached` , and the app ends the role-play.
  - The character never sees the next planned question.

- **Planning:**
  - Coding was missing for software jobs, because the JD analysis left out "technical" (**T-62, T-89**).
  - Coding and design questions repeated (**T-90**), and the fix's duplicate check then flagged different problems that shared boilerplate wording (**T-91**).
  - Coding problems were too easy for the level (**T-101**). A technique per level is now picked in code, since the model ignored a list of options in the prompt.

6.6 Scoring and the "stronger version"

- **Invented experience (T-15, T-18, T-22, T-48, T-111, T-129):**
  - rewrites made up actions, turned "I'd use X" into "I built X", and stitched résumé facts into stories that never happened;
  - fix: grounding checks in code (words, numbers and impact claims must come from the résumé or answers);
  - an answer that missed the question gets a résumé outline with placeholders instead;
  - `stitched()` blocks tying a story to a résumé fact the candidate never connected it to, and since **T-129** that also holds across sentences when the story is off-subject.
- **Example values anchor the model (T-68):** the example JSON in `score.txt` showed `"result": false` and `"covered": false`, and the model copied them (0 of 30 and 0 of 429 credited). The scorer now quotes what the candidate said first, and the STAR check and covered points follow those verified quotes.
- **False gaps (T-72):** the scorer listed points the candidate had made as missing. Each gap is now re-decided in its own call and kept only with a quote that really is in the answers. **T-108** drops gaps that don't fit the question.
- **Weights (T-75):** the rubric weights came back as the model returned them (communication about 5% each). They are now fixed in code: role 70%, communication 30%.
- **Non-answers (T-33, T-61, T-112):**
  - "Thank you." scored 3.6, and a 12-word greeting scored 3.56;
  - code now catches very short answers and small talk;
  - a "didn't answer the question" verdict needs a second check before the answer scores 1.
- **Off-subject stories (T-114, T-118, T-120, T-128):**
  - a real story about a different project scored 3.67;
  - the match is now decided in code: the named subject must be a résumé item that the answer never mentions;
  - its first live tests produced false positives ("p95" was split apart, and "retry" didn't match "retries"), fixed in **T-120** and **T-128**.
- **Results in rewrites (T-107, T-121, T-123, T-126):** a stated result was replaced by a placeholder, repeated, dropped, or left next to a redundant placeholder. Each case is now handled in `fill_result`.

6.7 Browser and recorder

- **T-51:** Monaco's CDN loader defined a global `define()` that broke Excalidraw and MediaPipe. Fixed by bundling Monaco.
- **T-52:** MediaPipe sampling on a fixed 250 ms interval could freeze slow devices. Sampling now adapts.
- **T-60:** the panel voice replayed the intro, because a hook dependency changed on every turn.
- **T-82:** code typed during a spoken answer was lost, because a stale closure held the empty editor.

- **T-88:** Cancel still sent the answer, because of a flat 4 s silence window and buttons that moved.
- **T-83:** the browser voice spelled out "Mm-hm" letter by letter.

6.8 Security

- **T-124:** a scratch test file holding a live-site login token was committed by accident (`1d1a184`), because the commit staged a whole folder. It was removed in `60ae9fe` and `live*.json` is now ignored in git and in the API image.
- **T-125:** testing that token showed **tokens of deleted accounts were still accepted** until they expired. From the code, such a token could also have started untracked Fire work. `bcb5271` adds a user lookup on every request, so a deleted account's token gets 401 before any Fire call.
- **Hardening** (`d3427b2`): rate limits on login, signup and delete; `APP_ENV=production` refuses weak secrets, SQLite, a localhost CORS origin or a missing Fire key; `TRUST_PROXY` (ProxyFix); gunicorn.

6.9 Fire speech had no model (T-27)

- On 2026-10-09 Fire's `/v1/speech` returned 404 `PROVIDER_UNAVAILABLE`, which blocked Fire voice in Phase 6.
- By 2026-10-11 Fire offered speech models. `tts-kokoro` was pinned after a live probe that cost $0.000315.

---

7. Deployment

Plan versus reality.

- **The plan** (`DEPLOY.md` §1–2):
  - an Oracle Cloud Always Free ARM VM (2 cores / 12 GB) running Docker Compose with MySQL 8.4, the gunicorn API, a daily purge service and **Caddy** for automatic HTTPS;
  - DuckDNS for the hostname, with a $1 Oracle budget alert.
- **What happened:** Oracle refused the sign-up. The same stack runs on the **devbox** (§2b, **T-76**):
  - code in `/srv/intervox` ;
  - `docker compose -f docker-compose.yml -f deploy/compose.devbox.yml` (Caddy off, API only on `127.0.0.1:8000`);
  - nginx site `intervox` with a certbot certificate for **https://intervox.duckdns.org**.
- **Frontend: Vercel Hobby**, from the `frontend/` folder with `vercel.json` (page routes, security headers, cached assets). It auto-deploys `main` to **https://intervox-pi.vercel.app**. `VITE_API_URL` points at the DuckDNS API.
- **Security headers:** HSTS, `nosniff` , `X-Frame-Options: DENY` , Referrer-Policy and Permissions-Policy, on both nginx/Caddy and Vercel. The API host answers only `/api/*` ; everything else gets 404.
- **Backups:** root cron at 03:00 runs `deploy/backup.sh` , which writes to `/root/backups` and keeps 14 (**T-79/T-80**).
- **Deploy flow:** ticket `pending_review` with a JSON `deploy_plan` , then `pm approve T-n` by the user, then master pulls, rebuilds `api` (migrations run on start), checks `/api/health` and closes the ticket.

Safety and budget guards (`CLAUDE.md` , `config.py` , `services/limits.py`):

- **Monthly Fire budget.** `check_budget()` blocks new Fire work once the month's total reaches `FIRE_MONTHLY_BUDGET_USD`: $1 by default, $1.45 for October 2026 on the live site. Spend is tracked from each response's `meta.price.usd`, per user and in an anonymous tally.
- **Daily caps per user:**

Cap	Limit
Interview minutes	60
60-minute sessions	1
Plans	10
Retries	20
Decoder runs	5
Signed-out decoder	3 per person, 30 per day

- **The Fire key** lives only in the server's `.env` (root-only, mode 600). The browser never calls Fire.
- **No audio or video stored.** Clips are dropped after transcription, and debrief accepts no recordings.
- **Rate limits** on login, signup and delete; **production settings check** at startup; **401 for a deleted account's token** (**T-125**).
- **Privacy:** daily purge of expired documents, Delete my data, and revocable share links (`services/privacy.py`).
- **Admin page** (`/admin`) shows usage, spend and the budget left.

---

8. What's left and future plans

- **November 2026:** set the live `FIRE_MONTHLY_BUDGET_USD` back from 1.45 to 1, through a master deploy ticket (**T-76**, `CLAUDE.md`).
- **Not yet seen in a live session:** T-117, T-120 and T-126–T-130. Their tests pass with Fire mocked, but they only show up when the model words things a certain way.
- **Phase 11 (optional):** real-time voice with LiveKit (interruptions, peer practice). **Phase 12 (optional):** a Hume EVI emotion experiment (`CLAUDE.md`). Neither has started.
- **Oracle:** `DEPLOY.md` notes the stack can move to Oracle later with a backup and the same `.env`, "if Oracle ever works out".
- **Open tickets:** none as of 2026-10-13. All 114 InterVox tickets are closed.
- *Unclear:*
  - The sources don't record total Fire spend for October; the live database couldn't be read from this session. The rules check and run tickets log individual check costs (for example ~$0.034, ~$0.05, $0.0744, $0.0768).
  - Ticket numbers are shared across the devbox. The gaps in InterVox's list (T-1–T-11, T-28, T-50, T-59, T-67, T-110) aren't InterVox tickets in the database, and the sources don't say what happened to them.

---

9. Lessons learned

These are drawn from ticket root causes and the rules-check log.

1. **Enforce in code what must not happen.** Prompt-only rules (no praise, no invented experience, relevance) were broken until code checked them (**T-14, T-15, T-48, T-114**). When code made the decision, as with the coding technique per level (**T-101**) or the off-subject match (**T-114**), the behaviour became reliable.
2. **Example values and option lists anchor a small model.** Example JSON with `false` values was copied (**T-68**), and a list of options in a prompt was ignored (**T-101**).
3. **Mocked tests aren't enough; check live, cheaply.** Most rewrite and scoring bugs (T-44–T-48, T-105, T-107–T-130) were found in short live sessions costing about $0.01–0.08 each, after the mocked tests passed.
4. **Each fix can create a new edge case.** The off-subject rule (T-114) needed two follow-ups for false positives (T-120, T-128). The praise filter (T-14) later needed T-117. Live re-checks after each fix caught these.
5. **One bad item shouldn't fail the whole job.** The debrief report blocked on a single question (**T-105**) until reports learned to set aside what can't be scored (**T-93, T-105**).
6. **Fail loudly in operations scripts.** The backup reported success on failure until it used `pipefail` and a partial file (**T-80**).
7. **Watch what goes into a commit.** Staging a whole folder committed a token (**T-124**). Checking that token then found a real gap: deleted-account tokens were still accepted (**T-125**).
8. **Plan for the free tier to say no.** Oracle refused the sign-up. A host-agnostic compose file with a small override (`compose.devbox.yml`) moved the deploy to the devbox with almost no changes.
9. **Keep a human in the loop for anything with blast radius.** Deploys went through `pm approve` and the master agent.